

Mini RFC: Path Validation

April 5, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Overview

Address validation ensures that an endpoint cannot be used for a traffic amplification attack. In such an attack, a packet is sent to a server with spoofed source address information that identifies a victim. If a server generates more or larger packets in response to that packet, the attacker can use the server to send more data toward the victim than it would be able to send on its own.

Or more general solution: Source Address Validation [RFC 7039].

The primary defense against amplification attacks is verifying that a peer is able to receive packets at the transport address that it claims. Therefore, after receiving packets from an address that is not yet validated, an endpoint **MUST** limit the amount of data it sends to the unvalidated address to three times the amount of data received from that address. This limit on the size of responses is known as the anti-amplification limit.

Address validation is performed both during connection establishment and during connection migration.

2 Path Validation

Path validation is used by both peers during connection migration to verify reachability after a change of address. In path validation, endpoints test reachability between a specific local address and a specific peer address, where an address is the 2-tuple of IP address and port.

Path validation tests that packets sent on a path to a peer are received by that peer. Path validation is used to ensure that packets received from a migrating peer do not carry a spoofed source address.

Path validation does not validate that a peer can send in the return direction. Acknowledgments cannot be used for return path validation because they contain insufficient entropy and might be spoofed. Endpoints independently determine reachability on each direction of a path, and therefore return reachability can only be established by the peer.

Path validation can be used at any time by either endpoint. For instance, an endpoint might check that a peer is still in possession of its address after a period of quiescence.

An endpoint MAY include other frames with the PATH_CHALLENGE and PATH_RESPONSE frames used for path validation. In particular, an endpoint can include PADDING frames with a PATH_CHALLENGE frame for Path Maximum Transmission Unit Discovery (PMTUD). An endpoint can also include its own PATH_CHALLENGE frame when sending a PATH_RESPONSE frame.

Optional features: *PMTU Discover*

An endpoint uses a new connection ID for probes sent from a new local address. When probing a new path, an endpoint can ensure that its peer has an unused connection ID available for responses. Sending NEW_CONNECTION_ID and PATH_CHALLENGE frames in the same packet, if the peer's active_connection_id_limit permits, ensures that an unused connection ID will be available to the peer when sending a response.

An endpoint can choose to simultaneously probe multiple paths. The number of simultaneous paths used for probes is limited by the number of extra connection IDs its peer has previously supplied, since each new local address used for a probe requires a previously unused connection ID.

2.1 Initiating Path Validation

To initiate path validation, an endpoint sends a PATH_CHALLENGE frame containing an unpredictable payload on the path to be validated.

An endpoint MAY send multiple PATH_CHALLENGE frames to guard against packet loss. However, an endpoint SHOULD NOT send multiple PATH_CHALLENGE frames in a single packet.

An endpoint SHOULD NOT probe a new path with packets containing a PATH_CHALLENGE frame more frequently than it would send an Initial packet. This ensures that connection migration is no more load on a new path than establishing a new connection.

The endpoint MUST use unpredictable data in every PATH_CHALLENGE frame so that it can associate the peer's response with the corresponding PATH_CHALLENGE.

An endpoint MUST expand datagrams that contain a PATH_CHALLENGE frame to at least the smallest allowed maximum datagram size of 1200 bytes, unless the anti-amplification limit for the path does not permit sending a datagram of this size. Sending UDP datagrams of this size ensures that the network path from the endpoint to the peer can be used for QUIC.

When an endpoint is unable to expand the datagram size to 1200 bytes due to the anti-amplification limit, the path MTU will not be validated. To ensure that the path MTU is large enough, the endpoint MUST perform a

Only required if you implement *PMTU Discover*.

second path validation by sending a `PATH_CHALLENGE` frame in a datagram of at least 1200 bytes. This additional validation can be performed after a `PATH_RESPONSE` is successfully received or when enough bytes have been received on the path that sending the larger datagram will not result in exceeding the anti-amplification limit.

Unlike other cases where datagrams are expanded, endpoints **MUST** NOT discard datagrams that appear to be too small when they contain `PATH_CHALLENGE` or `PATH_RESPONSE`.

2.2 Path Validation Responses

On receiving a `PATH_CHALLENGE` frame, an endpoint **MUST** respond by echoing the data contained in the `PATH_CHALLENGE` frame in a `PATH_RESPONSE` frame. An endpoint **MUST NOT** delay transmission of a packet containing a `PATH_RESPONSE` frame unless constrained by congestion control.

A `PATH_RESPONSE` frame **MUST** be sent on the network path where the `PATH_CHALLENGE` frame was received. This ensures that path validation by a peer only succeeds if the path is functional in both directions. This requirement **MUST NOT** be enforced by the endpoint that initiates path validation, as that would enable an attack on migration.

An endpoint **MUST** expand datagrams that contain a `PATH_RESPONSE` frame to at least the smallest allowed maximum datagram size of 1200 bytes. This verifies that the path is able to carry datagrams of this size in both directions. However, an endpoint **MUST NOT** expand the datagram containing the `PATH_RESPONSE` if the resulting data exceeds the anti-amplification limit. This is expected to only occur if the received `PATH_CHALLENGE` was not sent in an expanded datagram.

An endpoint **MUST NOT** send more than one `PATH_RESPONSE` frame in response to one `PATH_CHALLENGE` frame. The peer is expected to send more `PATH_CHALLENGE` frames as necessary to evoke additional `PATH_RESPONSE` frames.

2.3 Successful and failed Path Validation

Path validation succeeds when a `PATH_RESPONSE` frame is received that contains the data that was sent in a previous `PATH_CHALLENGE` frame. A `PATH_RESPONSE` frame received on any network path validates the path on which the `PATH_CHALLENGE` was sent.

If an endpoint sends a `PATH_CHALLENGE` frame in a datagram that is not expanded to at least 1200 bytes and if the response to it validates the peer address, the path is validated but not the path MTU. As a result, the endpoint can now send more than three times the amount of data that has been received. However, the endpoint **MUST** initiate another path validation with an expanded datagram to verify that the path supports the required MTU.

Receipt of an acknowledgment for a packet containing a `PATH_CHALLENGE` frame is not adequate validation, since the acknowledgment can be spoofed by a malicious peer.

Path validation only fails when the endpoint attempting to validate the path abandons its attempt to validate the path.

Endpoints SHOULD abandon path validation based on a timer. When setting this timer, implementations are cautioned that the new path could have a longer round-trip time than the original. A value of three times the larger of the current PTO or the PTO for the new path (using `kInitialRtt`) is RECOMMENDED.

This timeout allows for multiple PTOs to expire prior to failing path validation, so that loss of a single `PATH_CHALLENGE` or `PATH_RESPONSE` frame does not cause path validation failure.

Note that the endpoint might receive packets containing other frames on the new path, but a `PATH_RESPONSE` frame with appropriate data is required for path validation to succeed.

When an endpoint abandons path validation, it determines that the path is unusable. This does not necessarily imply a failure of the connection – endpoints can continue sending packets over other paths as appropriate. If no paths are available, an endpoint can wait for a new path to become available or close the connection. *An endpoint that has no valid network path to its peer MAY signal this using the `NO_VIABLE_PATH` connection error, noting that this is only possible if the network path exists but does not support the required MTU.*

A path validation might be abandoned for other reasons besides failure. Primarily, this happens if a connection migration to a new path is initiated while a path validation on the old path is in progress.

A small joke: Send a error message telling your peer there is available path when there is no available path.

3 Packetization

3.1 Frames

3.1.1 `PATH_CHALLENGE` Frames

Endpoints can use `PATH_CHALLENGE` frames (type=`0x1a`) to check reachability to the peer and for path validation during connection migration.

```
PATH_CHALLENGE Frame {
    Type (i) = 0x1a,
    Data (64),
}
```

Figure 1: *`PATH_CHALLENGE` Frame Format*

`PATH_CHALLENGE` frames contain the following field:

- **Data:** This 8-byte field contains arbitrary data. Including 64 bits of entropy in a `PATH_CHALLENGE` frame ensures that it is easier to receive the packet than it is to guess the value correctly.

The recipient of this frame MUST generate a PATH_RESPONSE frame containing the same Data value.

3.1.2 PATH_RESPONSE Frames

A PATH_RESPONSE frame (type=0x1b) is sent in response to a PATH_CHALLENGE frame.

The format of a PATH_RESPONSE frame is identical to that of the PATH_CHALLENGE frame.

```
PATH_RESPONSE Frame {  
    Type (i) = 0x1b,  
    Data (64),  
}
```

Figure 2: *PATH_RESPONSE Frame Format*

If the content of a PATH_RESPONSE frame does not match the content of a PATH_CHALLENGE frame previously sent by the endpoint, the endpoint MAY generate a connection error of type `PROTOCOL_VIOLATION`.

4 Application Protocol

There is no new application protocol defined in this mini RFC.